

Detección y explotación de vulnerabilidades XSS almacenado y CSRF en DVWA

Configuración del entorno en DVWA

El primer paso en la identificación y explotación de vulnerabilidades requiere establecer un entorno de pruebas controlado, que simule con fidelidad un sistema vulnerable sin poner en riesgo entornos reales. Damn Vulnerable Web Application (DVWA) es una plataforma especialmente diseñada con fines educativos para demostrar fallas comunes en aplicaciones web. Su modo “Low” desactiva filtros de seguridad para permitir la identificación directa de vectores de ataque. Acceder a DVWA mediante un navegador local, iniciar sesión con credenciales predeterminadas (`admin/password`) y configurar el nivel de seguridad en “Low” permite evaluar el comportamiento de la aplicación sin sanitización ni validaciones robustas. Esta configuración deliberadamente insegura crea un terreno fértil para probar ataques como XSS almacenado y CSRF, los cuales son especialmente dependientes de la falta de controles del lado del servidor.

Identificación de vulnerabilidad XSS almacenado

El XSS almacenado implica insertar contenido malicioso que se persiste en la base de datos de la aplicación y se refleja de forma dinámica cada vez que es visualizado. En el módulo “XSS (Stored)” de DVWA, se detecta la presencia de campos de entrada como `Name` y `Message`, que son potenciales puntos de inyección. Estos datos se almacenan en el backend y se presentan nuevamente en la interfaz sin validación ni escape de caracteres especiales. Esta ausencia de codificación implica que cualquier contenido ingresado —incluyendo etiquetas `<script>`— será ejecutado directamente por el navegador del usuario que acceda posteriormente a la página.

Inyección de payload JavaScript persistente

El payload utilizado para probar esta vulnerabilidad consiste en un fragmento de JavaScript básico, como `<script>alert('XSS almacenado')</script>`. Este script, al ser ingresado en uno de los campos del formulario y enviado al servidor, queda almacenado sin ningún tipo de transformación. Posteriormente, cada vez que un usuario visualiza los comentarios guardados en esa sección del sitio, el script es ejecutado automáticamente por su navegador. Esta ejecución ocurre en el contexto de la sesión del usuario, con todos los privilegios que dicha sesión posea. A diferencia del XSS reflejado, esta variante no requiere interacción directa con el atacante, lo que amplifica su alcance y persistencia. Puede ser utilizado para robar cookies, redirigir usuarios a sitios maliciosos o cargar otros scripts externos sin conocimiento del usuario.

Confirmación del comportamiento y análisis del impacto

Una vez almacenado el script, su ejecución automática confirma que la aplicación carece de mecanismos de validación de entrada, codificación de salida, y políticas de seguridad como Content-Security-Policy. Esta omisión representa una vulnerabilidad crítica, especialmente

en plataformas multiusuario, ya que permite a un atacante ejecutar código arbitrario en los navegadores de otros usuarios autenticados. El impacto puede escalar desde simples molestias visuales hasta la completa toma de control de cuentas o la difusión de malware persistente. La verificación visual de una alerta emergente generada por el payload confirma de forma inequívoca la vulnerabilidad.

Recomendación técnica frente a XSS almacenado

La mitigación efectiva del XSS almacenado requiere la implementación de múltiples capas defensivas. En primer lugar, toda entrada del usuario debe ser validada y saneada de forma estricta, especialmente en campos que se almacenarán y se reflejarán posteriormente. La codificación de salida debe adaptarse al contexto (HTML, atributo, JavaScript, URL) para evitar interpretaciones erróneas por parte del navegador. Es imperativo utilizar funciones de escape específicas en lugar de generalizaciones. Además, la implementación de cabeceras de seguridad como **Content-Security-Policy** puede prevenir la ejecución de scripts no autorizados, incluso si logran almacenarse. Frameworks modernos que incorporan plantillas seguras pueden ayudar a abstraer estos controles a nivel estructural.

Ataque CSRF orientado a cambio de contraseña

El ataque CSRF (Cross-Site Request Forgery) explota la confianza del servidor en la sesión autenticada del navegador. En este escenario, se simula un ataque mediante un archivo HTML local que contiene un formulario oculto preconfigurado para enviar una solicitud POST al endpoint de cambio de contraseña de DVWA. El formulario contiene valores predefinidos para **password_new** y **password_conf**, y se envía automáticamente al cargarse la página mediante un atributo **onload**. Dado que el navegador envía automáticamente las cookies de sesión activas al dominio correspondiente, el servidor interpreta la petición como legítima, aun cuando fue iniciada por una fuente maliciosa externa.

Ejecución del ataque desde el navegador local

Al abrir el archivo HTML malicioso en un navegador con una sesión activa en DVWA, el formulario se envía automáticamente. Esto provoca el cambio de contraseña del usuario autenticado sin su conocimiento ni consentimiento. La ausencia de tokens de verificación y la falta de comprobación del origen de la solicitud permiten que esta manipulación se concrete con éxito. Esta prueba evidencia una implementación vulnerable que no distingue entre peticiones originadas por el usuario y aquellas fabricadas por un tercero. El riesgo se amplifica cuando el usuario víctima posee privilegios elevados o acceso a funciones administrativas, ya que el atacante podría desencadenar cambios de configuración, eliminación de datos o elevación de privilegios.

Confirmación del resultado y análisis del riesgo

La verificación del cambio de contraseña confirma que la aplicación no implementa medidas anti-CSRF adecuadas. Este tipo de ataque es particularmente peligroso porque no requiere

la explotación de código malicioso ejecutado en el navegador, sino que se basa en el diseño deficiente de la lógica del servidor. La facilidad con la que se puede automatizar y ocultar este ataque mediante iframes, imágenes o scripts ocultos, lo convierte en una amenaza silenciosa pero devastadora. En contextos reales, un ataque CSRF puede ser la puerta de entrada para comprometer cuentas privilegiadas, alterar configuraciones críticas o facilitar ataques más avanzados como el spear phishing y la exfiltración de información.

Recomendación técnica frente a CSRF

Para mitigar el riesgo de CSRF, es esencial implementar tokens de verificación únicos por sesión y por formulario, los cuales deben ser verificados por el servidor en cada solicitud sensible. Estos tokens, inyectados dinámicamente en los formularios, aseguran que la petición provenga del contexto legítimo de la aplicación. Además, se deben aplicar restricciones a través de cabeceras como **Origin** y **Referer**, que permiten verificar si la solicitud proviene de un dominio autorizado. Otras buenas prácticas incluyen el uso de cookies con el atributo **SameSite=Strict** o **Lax**, evitando así su inclusión automática en solicitudes externas. Es recomendable también evitar el uso del método GET para operaciones que alteran el estado del sistema, limitando su uso exclusivamente a operaciones de lectura o navegación.

Informe técnico consolidado y reflexión profesional

Este ejercicio demuestra con claridad dos de las vulnerabilidades más críticas en aplicaciones web y su impacto potencial cuando no se aplican medidas de defensa en profundidad. La explotación exitosa de XSS almacenado y CSRF pone en evidencia deficiencias tanto en el saneamiento de datos como en la validación de flujos de autorización. El aprendizaje obtenido resalta la importancia de combinar validación de entradas, codificación de salidas, tokens de seguridad, políticas de contenido y buenas prácticas en el diseño de formularios y sesiones. La seguridad de una aplicación web no debe ser una fase posterior al desarrollo, sino una preocupación transversal desde el análisis de requisitos hasta la fase de mantenimiento. Prevenir estas vulnerabilidades es responsabilidad directa del equipo de desarrollo, pero también del equipo de QA, los arquitectos de software y los auditores de seguridad. Comprender estos vectores de ataque permite construir sistemas más resilientes, confiables y preparados para enfrentar un entorno digital cada vez más hostil.